# Capítulo 12 - Agente de Aprendizagem (Learning Agent)

## 12.1. Introdução ao Agente de Aprendizagem

O **Agente de Aprendizagem** é o componente de melhoria contínua do Projeto Quimera. Ele **coleta feedback** sobre o desempenho do sistema e **retreina modelos** para melhorar a eficácia ao longo do tempo.

### 12.1.1. Responsabilidades do Agente de Aprendizagem

1. **Coletar dados de treinamento:**
   - Mensagens que receberam resposta (positivas)
   - Mensagens que não receberam resposta (negativas)
   - Feedback explícito do usuário
2. **Processar feedback:**
   - Rotular exemplos (bom/ruim)
   - Identificar padrões de sucesso/falha
   - Calcular métricas de performance
3. **Retreinar modelos:**
   - Agente Analista (classificação de perfis)
   - Motor de Persuasão (seleção de estratégia)
   - Agente Escriba (geração de mensagens)
4. **Avaliar melhorias:**
   - A/B testing de modelos novos vs. antigos
   - Métricas de performance (taxa de resposta, conversão)
   - Rollback se performance piorar
5. **Otimizar hiperparâmetros:**
   - Ajustar pesos de pontuação
   - Ajustar thresholds de classificação
   - Ajustar temperatura de LLMs

### 12.1.2. Por que Aprendizagem Contínua?

**Problema:** O que funciona hoje pode não funcionar amanhã.

- **Mudanças no público:** Preferências e linguagem evoluem
- **Saturação:** Abordagens que funcionavam ficam "batidas"
- **Contexto local:** O que funciona em uma cidade pode não funcionar em outra
- **Personalização:** Cada usuário tem estilo único

**Solução:** Sistema que aprende e se adapta continuamente.

---

# 12.2. Tipos de Aprendizagem

## 12.2.1. Aprendizagem Supervisionada

**Definição:** Aprender com exemplos rotulados (bom/ruim).

**Exemplo:**

Plain Text

```
Entrada: Perfil + Mensagem
Rótulo: Recebeu resposta? (Sim/Não)

Treinar modelo para prever: "Esta mensagem receberá resposta?"
```

**Aplicações no Quimera:**

- Classificar perfis (interessante/não interessante)
- Prever taxa de resposta de uma mensagem
- Classificar sentimento de respostas

## 12.2.2. Aprendizagem por Reforço (Reinforcement Learning)

**Definição:** Aprender através de tentativa e erro, recebendo recompensas.

**Exemplo:**

Plain Text

```
Ação: Enviar mensagem com estratégia X
Resultado: Recebeu resposta positiva
Recompensa: +10 pontos

Ação: Enviar mensagem com estratégia Y
Resultado: Não recebeu resposta
```

```
Recompensa: -5 pontos

Aprender: Estratégia X é melhor que Y para este tipo de perfil
```

**Aplicações no Quimera:**

- Selecionar melhor princípio de persuasão
- Escolher melhor horário para enviar mensagem
- Otimizar frequência de mensagens

### 12.2.3. RLHF (Reinforcement Learning from Human Feedback)

**Definição:** Aprendizagem por reforço guiada por feedback humano.

**Processo:**

1. Sistema gera múltiplas mensagens
2. Humano ranqueia mensagens (melhor → pior)
3. Modelo aprende preferências humanas
4. Sistema gera mensagens melhores

**Por que RLHF?**

- Qualidade subjetiva (o que é uma "boa" mensagem?)
- Nuances culturais e contextuais
- Alinhamento com valores éticos

---

## 12.3. Coleta de Dados de Treinamento

### 12.3.1. Fontes de Dados

#### 1. Dados Implícitos (Automáticos)

Coletados automaticamente do sistema:

Python

```python
# Exemplo de episódio de treinamento
episodio = {
    'perfil_id': 12345,
    'perfil_features': {
        'idade': 25,
        'interesses': ['fotografia', 'viagens'],
        'personalidade': {'extroversao': 0.7, 'abertura': 0.8},
```

```
        'pontuacao_interessante': 85,
        'pontuacao_bonita': 78,
        'pontuacao_solteira': 90
    },
    'estrategia_usada': {
        'principio': 'afeicao',
        'tom': 'amigavel',
        'gatilhos': ['validacao', 'pertencimento']
    },
    'mensagem_enviada': "Adorei suas fotos de viagem! Também sou
apaixonado...",
    'resultado': {
        'recebeu_resposta': True,
        'tempo_resposta_horas': 3.5,
        'sentimento_resposta': 'positivo',
        'nivel_interesse': 'alto',
        'score_interesse': 85
    },
    'recompensa': 10  # Calculada baseada no resultado
}
```

**2. Dados Explícitos (Feedback Humano)**

Fornecidos pelo usuário:

Python

```python
# Interface para feedback
def coletar_feedback_mensagem(mensagem_id):
    """
    Permite usuário avaliar mensagem gerada.

    Interface:
    - Escala 1-5 estrelas
    - Comentário opcional
    - Sugestão de melhoria
    """
    feedback = {
        'mensagem_id': mensagem_id,
        'avaliacao': 4,  # 1-5
        'comentario': "Boa mensagem, mas poderia ser mais específica",
        'sugestao': "Mencionar algo sobre a foto específica, não só 'fotos'"
    }

    salvar_feedback(feedback)
```

## 12.3.2. Estrutura da Tabela de Episódios

SQL

```sql
CREATE TABLE episodios_treinamento (
    id BIGSERIAL PRIMARY KEY,
    perfil_id BIGINT REFERENCES perfis(id),
    perfil_features JSONB NOT NULL,
    estrategia_usada JSONB NOT NULL,
    mensagem_enviada TEXT NOT NULL,
    recebeu_resposta BOOLEAN DEFAULT FALSE,
    tempo_resposta_segundos INTEGER,
    sentimento_resposta VARCHAR(20),
    nivel_interesse VARCHAR(20),
    score_interesse INTEGER,
    recompensa FLOAT,
    feedback_usuario JSONB,
    criado_em TIMESTAMP DEFAULT NOW(),

    INDEX idx_recebeu_resposta (recebeu_resposta),
    INDEX idx_recompensa (recompensa DESC)
);
```

---

# 12.4. Cálculo de Recompensas

## 12.4.1. Função de Recompensa

Python

```python
def calcular_recompensa(resultado):
    """
    Calcula recompensa baseada no resultado da mensagem.

    Recompensa = soma ponderada de múltiplos fatores

    Returns:
        float: Recompensa (-10 a +20)
    """
    recompensa = 0

    # 1. Recebeu resposta? (+5 ou -5)
    if resultado['recebeu_resposta']:
        recompensa += 5
    else:
        recompensa -= 5
```

```python
    # 2. Tempo de resposta (0 a +5)
    if resultado['recebeu_resposta']:
        tempo_horas = resultado['tempo_resposta_horas']
        if tempo_horas < 1:
            recompensa += 5
        elif tempo_horas < 6:
            recompensa += 3
        elif tempo_horas < 24:
            recompensa += 1

    # 3. Sentimento da resposta (0 a +5)
    if resultado['sentimento_resposta'] == 'positivo':
        recompensa += 5
    elif resultado['sentimento_resposta'] == 'neutro':
        recompensa += 2
    elif resultado['sentimento_resposta'] == 'negativo':
        recompensa -= 3

    # 4. Nível de interesse (0 a +5)
    if resultado['nivel_interesse'] == 'alto':
        recompensa += 5
    elif resultado['nivel_interesse'] == 'medio':
        recompensa += 2

    # 5. Feedback do usuário (se houver) (-5 a +5)
    if resultado.get('feedback_usuario'):
        avaliacao = resultado['feedback_usuario']['avaliacao']
        recompensa += (avaliacao - 3)  # 1→-2, 3→0, 5→+2

    return recompensa
```

### 12.4.2. Exemplos de Recompensas

Python

```python
# Exemplo 1: Sucesso total
resultado1 = {
    'recebeu_resposta': True,
    'tempo_resposta_horas': 0.5,
    'sentimento_resposta': 'positivo',
    'nivel_interesse': 'alto'
}
recompensa1 = calcular_recompensa(resultado1)
# = 5 (resposta) + 5 (rápido) + 5 (positivo) + 5 (alto) = 20

# Exemplo 2: Sem resposta
resultado2 = {
```

```
    'recebeu_resposta': False
}
recompensa2 = calcular_recompensa(resultado2)
# = -5 (sem resposta) = -5

# Exemplo 3: Resposta negativa
resultado3 = {
    'recebeu_resposta': True,
    'tempo_resposta_horas': 12,
    'sentimento_resposta': 'negativo',
    'nivel_interesse': 'baixo'
}
recompensa3 = calcular_recompensa(resultado3)
# = 5 (resposta) + 1 (12h) - 3 (negativo) + 0 (baixo) = 3
```

---

## 12.5. Retreinamento de Modelos

### 12.5.1. Retreinamento do Agente Analista

**Objetivo:** Melhorar classificação de perfis (interessante/bonita/solteira).

Python

```python
def retreinar_modelo_analista():
    """
    Retreina modelo de classificação de perfis.

    Usa episódios onde:
    - Perfis que receberam resposta positiva → exemplos positivos
    - Perfis que não receberam resposta → exemplos negativos
    """
    # 1. Coletar dados de treinamento
    query = """
        SELECT
            perfil_features,
            CASE
                WHEN recebeu_resposta AND sentimento_resposta = 'positivo'
THEN 1
                ELSE 0
            END as label
        FROM episodios_treinamento
        WHERE criado_em > NOW() - INTERVAL '30 days'
    """

    dados = executar_query(query)
```

```python
    # 2. Preparar features e labels
    X = [ep['perfil_features'] for ep in dados]
    y = [ep['label'] for ep in dados]

    # 3. Treinar novo modelo
    from sklearn.ensemble import RandomForestClassifier

    modelo_novo = RandomForestClassifier(n_estimators=100)
    modelo_novo.fit(X, y)

    # 4. Avaliar performance
    from sklearn.model_selection import cross_val_score

    scores = cross_val_score(modelo_novo, X, y, cv=5)
    acuracia_media = scores.mean()

    print(f"Acurácia do novo modelo: {acuracia_media:.2%}")

    # 5. Comparar com modelo atual
    modelo_atual = carregar_modelo_atual('analista')
    scores_atual = cross_val_score(modelo_atual, X, y, cv=5)
    acuracia_atual = scores_atual.mean()

    print(f"Acurácia do modelo atual: {acuracia_atual:.2%}")

    # 6. Decidir se atualizar
    if acuracia_media > acuracia_atual:
        print("✅ Novo modelo é melhor! Atualizando...")
        salvar_modelo(modelo_novo, 'analista', versao='v2')
        return True
    else:
        print("❌ Modelo atual é melhor. Mantendo.")
        return False
```

### 12.5.2. Retreinamento do Motor de Persuasão

**Objetivo:** Melhorar seleção de princípio de persuasão.

Python

```python
def retreinar_motor_persuasao():
    """
    Retreina seletor de princípio de persuasão.

    Aprende: Qual princípio funciona melhor para cada tipo de perfil?
    """
    # 1. Coletar dados
```

```
    query = """
        SELECT
            perfil_features,
            estrategia_usada->>'principio' as principio,
            recompensa
        FROM episodios_treinamento
        WHERE recebeu_resposta = TRUE
        AND criado_em > NOW() - INTERVAL '30 days'
    """

    dados = executar_query(query)

    # 2. Agrupar por tipo de perfil e princípio
    from collections import defaultdict

    performance_por_principio = defaultdict(list)

    for ep in dados:
        perfil_tipo = classificar_tipo_perfil(ep['perfil_features'])
        principio = ep['principio']
        recompensa = ep['recompensa']

        chave = (perfil_tipo, principio)
        performance_por_principio[chave].append(recompensa)

    # 3. Calcular recompensa média por combinação
    melhores_principios = {}

    for (perfil_tipo, principio), recompensas in
performance_por_principio.items():
        recompensa_media = sum(recompensas) / len(recompensas)

        if perfil_tipo not in melhores_principios:
            melhores_principios[perfil_tipo] = []

        melhores_principios[perfil_tipo].append({
            'principio': principio,
            'recompensa_media': recompensa_media,
            'num_exemplos': len(recompensas)
        })

    # 4. Ordenar por recompensa
    for perfil_tipo in melhores_principios:
        melhores_principios[perfil_tipo].sort(
            key=lambda x: x['recompensa_media'],
            reverse=True
        )
```

```python
    # 5. Atualizar regras de seleção
    salvar_regras_persuasao(melhores_principios)

    return melhores_principios


def classificar_tipo_perfil(features):
    """Classifica perfil em tipo (para agrupamento)."""
    # Simplificação: usar personalidade dominante
    personalidade = features['personalidade']

    if personalidade['extroversao'] > 0.7:
        return 'extrovertido'
    elif personalidade['abertura'] > 0.7:
        return 'aberto'
    elif personalidade['amabilidade'] > 0.7:
        return 'amigavel'
    else:
        return 'equilibrado'
```

### 12.5.3. Fine-tuning do Agente Escriba

**Objetivo:** Melhorar qualidade das mensagens geradas.

Python

```python
def fine_tuning_escriba():
    """
    Fine-tuning do LLM para gerar mensagens melhores.

    Usa técnica de RLHF (Reinforcement Learning from Human Feedback).
    """
    # 1. Coletar exemplos de mensagens boas
    query = """
        SELECT
            perfil_features,
            estrategia_usada,
            mensagem_enviada,
            recompensa
        FROM episodios_treinamento
        WHERE recompensa > 10
        AND feedback_usuario->>'avaliacao' >= '4'
        ORDER BY recompensa DESC
        LIMIT 1000
    """

    exemplos_bons = executar_query(query)
```

```
    # 2. Coletar exemplos de mensagens ruins
    query_ruins = """
        SELECT
            perfil_features,
            estrategia_usada,
            mensagem_enviada,
            recompensa
        FROM episodios_treinamento
        WHERE recompensa < 0
        OR feedback_usuario->>'avaliacao' <= '2'
        ORDER BY recompensa ASC
        LIMIT 500
    """

    exemplos_ruins = executar_query(query_ruins)

    # 3. Preparar dataset de fine-tuning (formato OpenAI)
    dataset = []

    for ep in exemplos_bons:
        prompt = construir_prompt(ep['perfil_features'],
ep['estrategia_usada'])
        completion = ep['mensagem_enviada']

        dataset.append({
            'prompt': prompt,
            'completion': completion,
            'weight': 1.0  # Peso positivo
        })

    for ep in exemplos_ruins:
        prompt = construir_prompt(ep['perfil_features'],
ep['estrategia_usada'])
        completion = ep['mensagem_enviada']

        dataset.append({
            'prompt': prompt,
            'completion': completion,
            'weight': -0.5  # Peso negativo (evitar este tipo)
        })

    # 4. Salvar dataset
    import json

    with open('dataset_fine_tuning.jsonl', 'w') as f:
        for item in dataset:
            f.write(json.dumps(item) + '\n')
```

```python
    # 5. Executar fine-tuning (OpenAI API)
    from openai import OpenAI

    client = OpenAI()

    # Upload dataset
    file = client.files.create(
        file=open('dataset_fine_tuning.jsonl', 'rb'),
        purpose='fine-tune'
    )

    # Criar job de fine-tuning
    job = client.fine_tuning.jobs.create(
        training_file=file.id,
        model='gpt-4o-mini-2024-07-18'
    )

    print(f"Fine-tuning iniciado: {job.id}")

    return job.id
```

---

## 12.6. A/B Testing

### 12.6.1. Implementação de A/B Test

Python

```python
def executar_ab_test(modelo_a, modelo_b, num_testes=100):
    """
    Executa A/B test entre dois modelos.

    Args:
        modelo_a: Modelo atual (controle)
        modelo_b: Modelo novo (variação)
        num_testes: Número de testes a executar

    Returns:
        dict: Resultados do teste
    """
    resultados_a = []
    resultados_b = []

    # Obter perfis para teste
    perfis = obter_perfis_qualificados(limite=num_testes * 2)
```

```
    # Dividir em dois grupos (aleatório)
    import random
    random.shuffle(perfis)

    grupo_a = perfis[:num_testes]
    grupo_b = perfis[num_testes:num_testes*2]

    # Testar modelo A
    for perfil in grupo_a:
        estrategia = modelo_a.gerar_estrategia(perfil)
        mensagem = gerar_mensagem(perfil, estrategia)
        resultado = enviar_e_monitorar(perfil, mensagem)
        resultados_a.append(resultado)

    # Testar modelo B
    for perfil in grupo_b:
        estrategia = modelo_b.gerar_estrategia(perfil)
        mensagem = gerar_mensagem(perfil, estrategia)
        resultado = enviar_e_monitorar(perfil, mensagem)
        resultados_b.append(resultado)

    # Calcular métricas
    taxa_resposta_a = sum(r['recebeu_resposta'] for r in resultados_a) /
len(resultados_a)
    taxa_resposta_b = sum(r['recebeu_resposta'] for r in resultados_b) /
len(resultados_b)

    recompensa_media_a = sum(r['recompensa'] for r in resultados_a) /
len(resultados_a)
    recompensa_media_b = sum(r['recompensa'] for r in resultados_b) /
len(resultados_b)

    # Teste de significância estatística
    from scipy.stats import ttest_ind

    _, p_value = ttest_ind(
        [r['recompensa'] for r in resultados_a],
        [r['recompensa'] for r in resultados_b]
    )

    # Decidir vencedor
    if p_value < 0.05:  # Significância estatística
        if recompensa_media_b > recompensa_media_a:
            vencedor = 'B'
            melhoria = ((recompensa_media_b - recompensa_media_a) /
recompensa_media_a) * 100
        else:
```

```
            vencedor = 'A'
            melhoria = 0
    else:
        vencedor = 'Empate (não significativo)'
        melhoria = 0

    return {
        'modelo_a': {
            'taxa_resposta': taxa_resposta_a,
            'recompensa_media': recompensa_media_a
        },
        'modelo_b': {
            'taxa_resposta': taxa_resposta_b,
            'recompensa_media': recompensa_media_b
        },
        'p_value': p_value,
        'vencedor': vencedor,
        'melhoria_percentual': melhoria
    }
```

---

# 12.7. Otimização de Hiperparâmetros

## 12.7.1. Grid Search

Python

```python
def otimizar_hiperparametros():
    """
    Otimiza hiperparâmetros do sistema usando grid search.

    Hiperparâmetros a otimizar:
    - Pesos de pontuação (interessante, bonita, solteira)
    - Threshold de qualificação
    - Temperatura do LLM
    """
    from itertools import product

    # Definir grid de hiperparâmetros
    pesos_interessante = [0.3, 0.4, 0.5]
    pesos_bonita = [0.2, 0.3, 0.4]
    pesos_solteira = [0.2, 0.3, 0.4]
    thresholds = [60, 65, 70]
    temperaturas = [0.7, 0.8, 0.9]

    melhores_params = None
```

```
    melhor_performance = 0

    # Testar todas as combinações
    for params in product(pesos_interessante, pesos_bonita, pesos_solteira,
thresholds, temperaturas):
        peso_int, peso_bon, peso_sol, threshold, temp = params

        # Normalizar pesos
        soma_pesos = peso_int + peso_bon + peso_sol
        peso_int /= soma_pesos
        peso_bon /= soma_pesos
        peso_sol /= soma_pesos

        # Configurar sistema com estes parâmetros
        configurar_sistema({
            'pesos': {'interessante': peso_int, 'bonita': peso_bon,
'solteira': peso_sol},
            'threshold': threshold,
            'temperatura_llm': temp
        })

        # Executar teste (sample pequeno)
        resultados = executar_teste_sistema(num_perfis=20)

        # Calcular performance
        performance = resultados['recompensa_media']

        if performance > melhor_performance:
            melhor_performance = performance
            melhores_params = params

    print(f"Melhores parâmetros: {melhores_params}")
    print(f"Performance: {melhor_performance}")

    return melhores_params
```

---

## 12.8. Métricas de Aprendizagem

### 12.8.1. Métricas Importantes

Python

```
metricas_aprendizagem = {
    'episodios_coletados': 5000,
    'episodios_com_feedback': 150,
```

```
    'recompensa_media': 7.5,
    'taxa_resposta_geral': 0.32,  # 32%
    'taxa_conversao': 0.08,  # 8%
    'modelos_retreinados': 3,
    'melhoria_apos_retreinamento': 0.15,  # +15%
    'ab_tests_executados': 5,
    'ab_tests_vencidos_modelo_novo': 3
}
```

### 12.8.2. Dashboard de Aprendizagem

Python

```python
def gerar_dashboard_aprendizagem():
    """Gera dashboard com evolução do sistema."""
    # Métricas ao longo do tempo
    query = """
        SELECT
            DATE_TRUNC('week', criado_em) as semana,
            AVG(recompensa) as recompensa_media,
            AVG(CASE WHEN recebeu_resposta THEN 1 ELSE 0 END) as
taxa_resposta,
            COUNT(*) as num_episodios
        FROM episodios_treinamento
        GROUP BY semana
        ORDER BY semana
    """

    evolucao = executar_query(query)

    # Plotar evolução
    import matplotlib.pyplot as plt

    semanas = [e['semana'] for e in evolucao]
    recompensas = [e['recompensa_media'] for e in evolucao]

    plt.plot(semanas, recompensas)
    plt.title('Evolução da Recompensa Média')
    plt.xlabel('Semana')
    plt.ylabel('Recompensa Média')
    plt.savefig('evolucao_aprendizagem.png')

    return evolucao
```

## 12.9. Considerações Éticas

### 12.9.1. Viés nos Dados

**Problema:** Sistema pode aprender vieses indesejados.

**Exemplo:**

- Se sistema aprende que mensagens mais "agressivas" têm taxa de resposta maior, pode gerar mensagens inapropriadas

**Solução:**

- Validação humana regular
- Filtros de segurança
- Métricas de qualidade além de taxa de resposta

### 12.9.2. Overfitting

**Problema:** Sistema otimiza demais para curto prazo, prejudicando longo prazo.

**Exemplo:**

- Mensagens que geram resposta imediata mas conversas ruins

**Solução:**

- Recompensas de longo prazo (conversão, não apenas resposta)
- Regularização
- Validação em dados não vistos

---

## 12.10. Conclusão do Capítulo

Este capítulo cobriu o **Agente de Aprendizagem** em profundidade:

- **Tipos de Aprendizagem:** Supervisionada, por Reforço, RLHF
- **Coleta de Dados:** Implícita e explícita, estrutura de episódios
- **Função de Recompensa:** 5 fatores, escala -10 a +20
- **Retreinamento:** Analista, Motor de Persuasão, Escriba
- **A/B Testing:** Comparação estatística de modelos
- **Otimização:** Grid search de hiperparâmetros
- **Métricas:** Evolução ao longo do tempo
- **Ética:** Viés e overfitting

Este é o último capítulo dos **7 agentes** do Projeto Quimera. Nos próximos capítulos, cobriremos tópicos complementares como segurança, testes e deployment.

**Próximo Capítulo:** Capítulo 13 - Segurança e Anti-Detecção

---

**Referências:**


- Reinforcement Learning: An Introduction (Sutton & Barto): http://incompleteideas.net/book/the-book.html
- RLHF: Training language models to follow instructions with human feedback: https://arxiv.org/abs/2203.02155
- A/B Testing Guide: https://www.optimizely.com/optimization-glossary/ab-testing/
- Hyperparameter Optimization: https://scikit-learn.org/stable/modules/grid_search.html